Tugas Besar 1 Inteligensi Artifisial

IF3170 Inteligensi Artifisial

Optimasi Pemuatan Paket pada Peti Kemas Truk Menggunakan Algoritma *Local Search*

Semester 5 Tahun 2026



Disusun oleh:

13524011 - Muhammad Iqbal Raihan

13524041 - Nathan Adhika Santosa

13524053 - Muhammad Haris Putra Sulastianto

Labotorium Inteligensi Artifisial

Teknik Informatika

Institut Teknologi Bandung

Daftar Isi

1 Deskripsi Persoalan **3**

1.1 Latar Belakang dan Deskripsi Persoalan 3

1.1.1 Fungsi Objektif dan Batasan (*Constraints*) 3

1.1.2 Representasi *State* dan Transisi *Neighbor* 4

1.2 Landasan Teori Algoritma *Local Search* 4

1.2.1 *Classical Search* vs *Local Search* 4

1.2.2 *State Value*, *Successor*, dan *Neighbor* 5

1.2.3 *Hill-Climbing Search* 5

1.2.4 *Simulated Annealing* 6

1.2.5 *Genetic Algorithm* 6

2 Pembahasan **7**

2.1 Perancangan Sistem 7

2.1.1 Representasi *State* 7

2.1.2 Rancangan *Initial State* dan *Neighbor* 8

2.2 Implementasi Program 8

2.2.1 Kelas Data (`foundation.py`) 8

2.2.2 Kelas `State` (`foundation.py`) 9

2.2.3 Kelas `HillClimbing` (`hill_climbing.py`) 16

2.3 Implementasi Simulated Annealing 20

2.3.1 Kelas dan pembangkitan tetangga 20

2.3.2 Aturan penerimaan dan penyimpanan hasil 21

2.4 Implementasi Genetic Algorithm 22

2.4.1 Representasi individu dan fitness 22

2.4.2 Seleksi, crossover, mutasi, dan replacement 22

2.5 Visualisasi State dan Hasil Pencarian 23

2.6 Hasil dan Analisis Eksperimen 23

2.6.1 Metodologi 23

2.6.2 Hill-Climbing 23

2.6.3 Simulated Annealing 25

2.6.4 Genetic Algorithm 26

3 Kesimpulan dan Saran **28**

3.1 Kesimpulan 28
3.2 Saran 29

4 Lampiran **29**
4.1 Lampiran Form Penggunaan Generative AI 29
4.2 Pranala Ke Repositori 31
4.3 Pembagian Tugas 31

Referensi **32**

1 Deskripsi Persoalan

1.1 Latar Belakang dan Deskripsi Persoalan

Dalam industri logistik darat, optimalisasi pemuatan barang ke dalam peti kemas (*Container Loading Problem* atau *3D Bin Packing Problem*) merupakan permasalahan kombinatorial yang krusial untuk memaksimalkan nilai ekonomi pengiriman. Pada studi kasus *Mejiro Express*, permasalahan berfokus pada penentuan konfigurasi penempatan paket berbentuk kubus atau balok ke dalam sebuah truk peti kemas sedemikian rupa sehingga total nilai paket yang diangkut menjadi maksimal tanpa melanggar batasan fisik peti kemas.

Permasalahan ini melibatkan dua entitas utama, yaitu paket (`Package`) dan truk peti kemas (`Truck`):

1. **Paket (`Package`):** Setiap paket memiliki atribut penanda unik (`id`), dimensi ukuran (`w`, `l`, `h`), orientasi sumbu (`Orientation` dengan 6 kemungkinan rotasi), nilai harga (`Value`), berat (`Weight`), status barang pecah-belah (`IsFragile`), serta perkiraan waktu kedatangan (`ETA`). Tidak semua paket wajib dimasukkan ke dalam truk; paket yang tidak dimuat akan berada sepenuhnya di luar truk.
2. **Truk (`Truck`):** Peti kemas truk direpresentasikan sebagai ruang tiga dimensi diskrit dengan ukuran lebar (`w`), panjang (`l`), dan tinggi (`h`), serta batas kapasitas angkut berat maksimum (`MaxCapacity`).

1.1.1 Fungsi Objektif dan Batasan (*Constraints*)

Tujuan utama (*objective function*) dari pencarian solusi adalah memaksimalkan total `Value` dari seluruh paket yang berada di dalam peti kemas truk. Konfigurasi penempatan paket dinyatakan sah (*valid*) apabila memenuhi seluruh batasan keras (*hard constraints*) berikut:

1. **Batas Dinding dan Tumpang Tindih:** Seluruh bagian paket yang dimuat harus berada di dalam dimensi ruang truk (tidak menembus dinding peti kemas) dan tidak boleh saling bertabrakan (*overlap*) pada koordinat tiga dimensi yang sama.
2. **Aturan Penopang (*Non-Floating*):** Paket tidak boleh melayang di udara. Misalkan paket i setelah diorientasikan memiliki posisi sudut kiri-bawah-belakang (x_i, y_i, z_i) dan dimensi efektif (w_i, d_i, h_i), maka bidang alas paket i adalah:

   $$B_i = \{(x, y, z_i) \mid x_i \leq x < x_i + w_i,\ y_i \leq y < y_i + d_i\}$$

   Paket i dikatakan tidak melayang apabila minimal terdapat satu titik pada alas B_i yang memiliki penopang (*support*), yaitu berada tepat di lantai truk ($z_i = 0$) atau terdapat paket lain j yang permukaan atasnya berada tepat di bawah titik tersebut ($z_j + h_j = z_i$).
3. **Aturan Barang Pecah-Belah (*Fragile*):** Paket dengan atribut `IsFragile = True` tidak boleh berada di bawah paket lain atau dijadikan sebagai penopang (*support*).

4. **Kapasitas Berat Maksimum:** Total berat (`Weight`) seluruh paket di dalam truk tidak boleh melebihi `MaxCapacity`.

1.1.2 Representasi *State* dan Transisi *Neighbor*

Pencarian dimulai dari keadaan awal (*initial state*) yang dibangkitkan secara acak (*random*). Pada setiap iterasi, pembangkitan keadaan tetangga (*neighbor state*) dilakukan melalui salah satu dari tiga operasi transisi berikut:

1. Menukar posisi dua buah paket, baik yang berada di dalam maupun di luar truk (*Swap*).
2. Memindahkan sebuah paket ke koordinat (x, y, z) yang berbeda di dalam truk (*Move*).
3. Merotasi orientasi paket sebesar 90° terhadap sumbu x, y, atau z (*Rotate*).

1.2 Landasan Teori Algoritma *Local Search*

1.2.1 *Classical Search* vs *Local Search*

Pada permasalahan pencarian klasik (*classical search*), ruang pencarian dieksplorasi secara sistematis dengan menyimpan jejak langkah di memori karena solusi akhirnya berupa urutan aksi menuju tujuan (*sequence of actions* atau *path to goal*). Sebaliknya, pada banyak permasalahan optimasi kombinatorial, jalur menuju tujuan tidaklah relevan (*path to goal is irrelevant*). Hal yang dipentingkan hanyalah konfigurasi keadaan akhir (*final state*).

Algoritma *local search* bekerja menggunakan formulasi keadaan lengkap (*complete state formulation*), di mana algoritma hanya menyimpan satu keadaan saat ini (*single current state*) dan tidak menyimpan jalur maupun biaya jalur (*path cost*). Pencarian dilakukan dengan berpindah dari *current state* menuju keadaan tetangganya (*neighbor state*). Kualitas setiap *state* dinilai berdasarkan nilai keadaan (*state value*) menggunakan fungsi objektif (*objective function*) yang ingin dimaksimalkan, atau fungsi biaya heuristik (*heuristic cost function*) yang ingin diminimalkan.

Secara konseptual, *local search* mengeksplorasi sebuah lanskap ruang keadaan (*state-space landscape*) yang memiliki dua komponen: lokasi (*location*) yang didefinisikan oleh *state*, dan ketinggian (*elevation*) yang didefinisikan oleh nilai fungsi objektif atau fungsi biaya heuristik. Tujuan utama pencarian ini adalah menemukan titik tertinggi secara keseluruhan (*global maximum* atau *global optimum*). Namun, bergantung pada posisi *initial state*, pencarian berisiko terjebak pada beberapa kondisi lanskap berikut:

- ***Local Maximum*:** Puncak yang lebih tinggi dari seluruh keadaan tetangga di sekitarnya, tetapi lebih rendah daripada *global maximum*.
- ***"Flat" Local Maximum*:** Area puncak datar di mana seluruh tetangga memiliki nilai yang sama dengan *current state* dan tidak ada jalan menanjak lebih lanjut.
- ***Shoulder*:** Area datar (*plateau*) yang masih memiliki jalan menanjak menuju puncak yang lebih tinggi (*global maximum*) apabila ditelusuri lebih lanjut.

1.2.2 *State Value*, *Successor*, dan *Neighbor*

Dalam mekanisme transisi *local search*, fungsi suksesor (*successor function*) membangkitkan keadaan-keadaan yang mungkin dicapai dengan menerapkan satu aksi legal pada *current state*. Pendefinisian *neighbor* dari fungsi suksesor tersebut terbagi menjadi dua pendekatan:

1. ***Neighbor* sebagai *Highest-Valued Successor*:** Fungsi suksesor mengembalikan seluruh kemungkinan keadaan yang dapat dibangkitkan dari *current state*, lalu memilih suksesor dengan nilai tertinggi (apabila terdapat lebih dari satu suksesor terbaik yang bernilai sama, pemilihan dilakukan secara acak di antara himpunan suksesor terbaik tersebut).
2. ***Neighbor* sebagai *Random Successor*:** Fungsi suksesor hanya mengembalikan satu keadaan suksesor secara acak dari *current state* tanpa mengevaluasi seluruh kemungkinan suksesor.

1.2.3 *Hill-Climbing Search*

Hill-Climbing Search adalah algoritma perulangan yang dimulai dari suatu *initial state* acak dan terus-menerus bergerak ke arah peningkatan nilai (*increasing objective value*) atau penurunan biaya (*decreasing cost*). Pencarian akan berhenti (*terminate*) ketika mencapai suatu puncak (*peak*) di mana tidak ada tetangga yang memiliki nilai lebih tinggi. Terdapat empat varian utama yang digunakan:

- ***Steepest Ascent Hill-Climbing*:** Pada setiap iterasi, algoritma memilih *neighbor* yang merupakan *highest-valued successor* dari *current state*. Apabila tidak ada tetangga yang nilainya lebih tinggi (`neighbor.VALUE` $\leq$ `current.VALUE`), maka pencarian berhenti (termasuk saat menemui area datar/*flat*) dan mengembalikan *current state*.
- ***Hill-Climbing with Sideways Move*:** Mengatasi masalah berhentinya pencarian pada area *shoulder* dengan mengizinkan perpindahan mendatar ketika `neighbor.VALUE` = `current.VALUE`, dan baru berhenti jika `neighbor.VALUE` < `current.VALUE`. Untuk mencegah perulangan tak hingga pada *"flat" local maximum*, diberikan batas maksimum jumlah langkah mendatar berturut-turut (*limit on number of consecutive sideways moves*).
- ***Random Restart Hill-Climbing*:** Menjalankan serangkaian pencarian *Hill-Climbing* secara berurutan dari *initial state* acak yang baru setiap kali pencarian terjebak di *local optimum*, hingga tujuan ditemukan atau batas maksimum *restart* tercapai.
- ***Stochastic Hill-Climbing*:** Membangkitkan satu suksesor secara acak (*random successor*) pada setiap iterasi hingga mencapai batas $n_{\max}$ iterasi. Algoritma hanya berpindah ke *neighbor* tersebut jika nilainya lebih baik daripada keadaan saat ini (`neighbor.VALUE` > `current.VALUE`), dan mengabaikannya jika lebih buruk.

1.2.4 *Simulated Annealing*

Simulated Annealing (SA) menggabungkan efisiensi *hill-climbing* (yang tidak pernah melakukan *downhill moves* sehingga bersifat *incomplete*) dengan sifat lengkap (*complete*) dari *random walk*. Terinspirasi dari proses pemanasan dan pendinginan logam secara perlahan (*annealing*), algoritma ini merupakan versi *stochastic hill-climbing* yang mengizinkan beberapa langkah buruk (*downhill moves*) untuk keluar dari jebakan *local maxima*, namun frekuensi langkah buruk tersebut dikurangi secara bertahap seiring turunnya temperatur T berdasarkan fungsi jadwal waktu `schedule(t)`.

Pada setiap langkah waktu t, apabila $T = 0$ maka pencarian berhenti dan mengembalikan *current state*. Jika $T > 0$, algoritma memilih satu suksesor secara acak (`next`) dan menghitung selisih nilai objektif:

$$\Delta E = \texttt{next.VALUE} - \texttt{current.VALUE}$$

Aturan perpindahan ke keadaan `next` adalah sebagai berikut:

1. Jika $\Delta E > 0$ (*better neighbor*), perpindahan selalu dilakukan (`current` $\leftarrow$ `next`).
2. Jika $\Delta E < 0$ (*worse neighbor*), perpindahan tetap dapat dilakukan dengan probabilitas Boltzmann:
$$P(\text{terima}) = e^{\frac{\Delta E}{T}}$$
Perpindahan diterima apabila nilai probabilitas $e^{\Delta E/T}$ melampaui nilai pembanding (misalnya lebih besar dari random$(0, 1)$).

Saat temperatur T tinggi, peluang menerima langkah buruk besar sehingga pergerakan menyerupai *random walk*. Ketika T mendekati 0, keacakan berkurang drastis dan algoritma berperilaku seperti *stochastic hill-climbing*. Apabila T diturunkan dengan cukup lambat, *Simulated Annealing* akan menemukan *global optimum* dengan probabilitas mendekati 1.

1.2.5 *Genetic Algorithm*

Genetic Algorithm (GA) merupakan algoritma pencarian paralel berbasis populasi yang dikembangkan dari konsep *Local Beam Search* dan *Stochastic Beam Search*, di mana keadaan suksesor dibangkitkan dengan cara menggabungkan dua keadaan induk (*combining two parent states*) untuk saling meneruskan informasi yang bermanfaat. Tahapan utama dalam *Genetic Algorithm* meliputi:

1. **Populasi Awal (*Initial Population*) dan Representasi Individu:** Algoritma dimulai dengan k buah keadaan awal yang dibangkitkan secara acak (*population of k individuals*), di mana setiap individu (*state*) direpresentasikan sebagai barisan simbol/karakter pada alfabet berhingga yang mewakili variabel persoalan.
2. **Fungsi Evaluasi (*Fitness Function*):** Setiap individu dievaluasi menggunakan fungsi kebugaran (`FITNESS-FN`) yang memberikan nilai keadaan (*state value*) lebih tinggi bagi konfigurasi yang lebih baik.

3. **Seleksi Induk (*Selection*):** Memilih dua individu induk secara acak (`RANDOM-SELECTION`) dengan probabilitas pemilihan yang proporsional terhadap nilai *fitness* masing-masing (*roulette wheel selection*):

$$P(i) = \frac{F_i}{\sum_{j=1}^{k} F_j}$$

   Individu dengan nilai *fitness* tertinggi memiliki bagian peluang terbesar untuk terpilih sebagai induk.

4. **Persilangan (*Cross-Over / Reproduce*):** Mengombinasikan dua induk terpilih melalui fungsi `REPRODUCE(x, y)` pada suatu titik potong acak (*random crossover point*) untuk menghasilkan individu anak (*child/offspring*).

5. **Mutasi (*Mutation*):** Dengan probabilitas acak yang kecil (*small random probability*), individu anak dimodifikasi secara acak (`MUTATE`) pada titik mutasi acak (*random mutation point*). Generasi baru ini kemudian menggantikan populasi lama secara berulang hingga ditemukan individu yang cukup baik atau batas iterasi/waktu tercapai.

2 Pembahasan

2.1 Perancangan Sistem

Program disusun dalam beberapa modul yang memiliki tanggung jawab berbeda. Berkas `foundation.py` menyimpan representasi persoalan, fungsi objektif dasar, validasi *constraint*, pembuatan *initial state*, dan pembangkitan *neighbor*. Berkas `HC/hill_climbing.py`, `SA/simulated_annealing.py`, dan `GA/genetic_algorithm.py` masing-masing mengimplementasikan Hill-Climbing, Simulated Annealing, dan Genetic Algorithm. Berkas `visualization.py` menangani visualisasi state 3D dan grafik eksperimen, sedangkan `run_hc.py`, `run_sa.py`, dan `run_ga.py` menjalankan pengulangan eksperimen serta menyimpan `summary.json` dan gambar. Pemisahan ini memungkinkan komponen representasi dan validasi digunakan bersama oleh berbagai algoritma.

2.1.1 Representasi *State*

State disimpan sebagai *dictionary* `placements` yang memetakan `id` setiap paket ke objek `PackagePlacement` bila paket berada di dalam truk, atau ke `None` bila paket berada di luar truk. Objek `PackagePlacement` menyimpan koordinat bilangan bulat `x`, `y`, `z` (sudut kiri-bawah-belakang) dan `orientation` berupa *tuple* tiga huruf yang menyatakan dimensi paket yang sejajar dengan sumbu x, y, dan z. Misalnya `('l', 'w', 'h')` berarti sisi *length* sejajar sumbu x, sisi *width* sejajar sumbu y, dan sisi *height* sejajar sumbu z.

Representasi ini dipilih karena memetakan seluruh paket, baik yang di dalam maupun di luar truk, sesuai ketentuan spesifikasi. Selain itu, akses dan pembaruan penempatan satu paket cukup satu operasi, dan *state* mudah disalin sehingga setiap *neighbor* dibangun tanpa mengubah *state* asal.

2.1.2 Rancangan *Initial State* dan *Neighbor*

Operator *swap*, *move*, dan *rotate* mengikuti spesifikasi (lihat Bagian 1). Keputusan rancangan yang diambil pada implementasi adalah sebagai berikut.

- ***Initial state.*** Paket diurutkan secara acak, lalu masing-masing dicoba diletakkan di lantai truk ($z = 0$) dengan orientasi dan koordinat x, y acak. Penempatan yang tidak valid dibatalkan sehingga paket tersebut tetap di luar truk.
- ***Rotate.*** Satu langkah *rotate* memutar paket 90° terhadap satu sumbu saja (x, y, atau z). Rotasi ini setara dengan menukar dua huruf pada *tuple* orientasi, sehingga setiap orientasi memiliki tiga hasil rotasi.
- **Operator *insert.*** Operator tambahan untuk memasukkan paket dari luar truk ke posisi dan orientasi acak. Tanpa operator ini, paket di luar truk hanya dapat masuk melalui *swap*. Karena ruang koordinat truk besar, kandidat *insert* tidak dienumerasi seluruhnya, melainkan diambil secara acak sebanyak `insert_tries` percobaan per paket di luar truk.
- **Koordinat z terbatas.** *Move* dan *insert* hanya mencoba $z = 0$ atau z yang sama dengan permukaan atas paket lain di dalam truk, karena nilai z lain pasti menghasilkan paket melayang.
- **Pergeseran lokal.** *Move* menggeser x dan y paling jauh `max_shift` satuan (bawaan 2) agar *neighbor* tetap berada di sekitar *state* saat ini.
- **Dua mode pembangkitan.** Mode lengkap mengumpulkan seluruh kandidat *swap*, *move*, *rotate*, dan *insert* sehingga cocok untuk varian yang memilih *neighbor* terbaik. Mode *sampling* membangkitkan *neighbor* acak satu per satu dengan jenis aksi yang dipilih seragam, dan dipakai oleh *Stochastic Hill-Climbing*. Jenis aksi dibuat seimbang karena *move*, *rotate*, dan *swap* antar dua paket di dalam truk tidak mengubah nilai objektif (himpunan paket di dalam truk tetap), sehingga jika dipilih sebanding jumlah kandidatnya, sampel akan didominasi aksi yang tidak menghasilkan perbaikan.
- ***Resampling.*** Karena kandidat *insert* diambil secara acak, himpunan *neighbor* dapat berbeda pada setiap pembangkitan. Varian yang memakai mode lengkap karena itu mencoba membangkitkan ulang *neighbor* hingga `max_resamples` kali (bawaan 3) sebelum menyatakan pencarian terjebak. Konsekuensinya, pencarian berhenti tidak menjamin tercapainya *local optimum* mutlak terhadap seluruh kemungkinan penempatan.

2.2 Implementasi Program

2.2.1 Kelas Data (`foundation.py`)

Kelas `Package` dan `Truck` menyimpan atribut persoalan. Atribut `eta` dibaca dari masukan tetapi tidak dipakai oleh fungsi objektif maupun *constraint*.

Listing 1: Kelas `Package` dan `Truck`

```
1 @dataclass
```

```python
class Package:
    id: str
    w: int
    l: int
    h: int
    value: int
    weight: int
    is_fragile: bool
    eta: int

@dataclass
class Truck:
    w: int
    l: int
    h: int
    maxCapacity: int
```

Kelas `PackagePlacement` menyimpan penempatan satu paket. Metode `get_effective_dimensions` mengubah dimensi paket menjadi dimensi pada sumbu x, y, z sesuai orientasinya.

Listing 2: Kelas `PackagePlacement`

```python
@dataclass
class PackagePlacement:
    Class to store coordinate and orientation of a single package inside the truck
    x: int
    y: int
    z: int
    orientation: Tuple[str, str, str]

    def get_effective_dimensions(self, pkg: Package) -> Tuple[int, int, int]:
        dims = {'w': pkg.w, 'l': pkg.l, 'h': pkg.h}
        eff_x = dims[self.orientation[0]]
        eff_y = dims[self.orientation[1]]
        eff_z = dims[self.orientation[2]]
        return eff_x, eff_y, eff_z
```

2.2.2 Kelas `State` (`foundation.py`)

Metode `copy_state` membuat salinan *state* dengan menyalin setiap objek penempatan, sehingga *neighbor* dapat dimodifikasi tanpa memengaruhi *state* asal. Metode `get_objective_value` menjumlahkan nilai (`value`) seluruh paket yang berada di dalam truk, yaitu yang penempatannya bukan `None`. Metode ini hanya dipanggil pada *state* valid; karena setiap *neighbor* sudah disaring oleh `is_valid`, asumsi tersebut selalu terpenuhi selama pencarian.

Listing 3: `State`: salinan dan fungsi objektif

```python
@dataclass
class State:
    Class for each state condition of the truck at a certain time

    # Dictionary for package_id -> its placement
    placements: Dict[str, Optional[PackagePlacement]]

```

```
    def copy_state(self) -> 'State':
        return State(placements={
            pkg_id: copy.copy(placement) if placement is not None else None
            for pkg_id, placement in self.placements.items()
        })

    # Objective Function
    def get_objective_value(self, package_catalog: Dict[str, Package]) -> int:
        Assumption: Only called on a valid state (is_valid() == True)
        total = 0
        for pkg_id, placement in self.placements.items():
            if placement is not None:
                total += package_catalog[pkg_id].value
        return total
```

Pengecekan validitas (`is_valid`). Metode ini mengumpulkan batas ruang tiap paket di dalam truk pada ketiga sumbu, lalu memeriksa *constraint* pada Bagian 1 secara berurutan: kapasitas berat, batas dinding truk, tumpang tindih antarpaket, dan *support*. Pada pemeriksaan *support*, paket dengan $z > 0$ dianggap tertopang bila ada paket lain yang permukaan atasnya tepat berada pada z paket tersebut dan alasnya beririsan pada bidang xy. Jika paket penopang itu *fragile*, *state* langsung dinyatakan tidak valid. Pengecekan membandingkan setiap pasangan paket sehingga biayanya kuadratik terhadap jumlah paket di dalam truk.

Listing 4: Pengecekan validitas *state*

```
    # Constraint Checking
    def is_valid(self, truck: Truck, package_catalog: Dict[str, Package]) -> bool:
        placed = []
        total_weight = 0

        # Extract all package data inside the truck
        for pkg_id, pos in self.placements.items():
            if pos is not None:
                pkg = package_catalog[pkg_id]
                eff_x, eff_y, eff_z = pos.get_effective_dimensions(pkg)
                placed.append({
                    'id': pkg_id,
                    'x1': pos.x, 'x2': pos.x + eff_x,
                    'y1': pos.y, 'y2': pos.y + eff_y,
                    'z1': pos.z, 'z2': pos.z + eff_z,
                    'is_fragile': pkg.is_fragile
                })
                total_weight += pkg.weight

        # Check weight capacity
        if total_weight > truck.maxCapacity:
            return False

        for i, p1 in enumerate(placed):
            # Check truck boundaries (Out of bounds)
            if p1['x1'] < 0 or p1['x2'] > truck.w: return False
            if p1['y1'] < 0 or p1['y2'] > truck.l: return False
            if p1['z1'] < 0 or p1['z2'] > truck.h: return False

            # Check for overlapping packages
```

```
            for j in range(i + 1, len(placed)):
                p2 = placed[j]
                overlap_x = max(p1['x1'], p2['x1']) < min(p1['x2'], p2['x2'])
                overlap_y = max(p1['y1'], p2['y1']) < min(p1['y2'], p2['y2'])
                overlap_z = max(p1['z1'], p2['z1']) < min(p1['z2'], p2['z2'])

                if overlap_x and overlap_y and overlap_z:
                    return False

            # Check floating and fragile support
            if p1['z1'] > 0:
                has_valid_support = False
                for p2 in placed:
                    if p1['id'] != p2['id']:
                        # If p2's top is exactly at p1's bottom
                        if p2['z2'] == p1['z1']:
                            # Check 2D base intersection
                            overlap_x = max(p1['x1'], p2['x1']) < min(p1['x2'], p2['x2'])
                            overlap_y = max(p1['y1'], p2['y1']) < min(p1['y2'], p2['y2'])

                            if overlap_x and overlap_y:
                                # If a fragile item is used as support, INVALID!
                                if p2['is_fragile']:
                                    return False
                                has_valid_support = True

                if not has_valid_support:
                    return False # Floating without valid support

        return True
```

Rotasi 90° (`get_90deg_rotations`). Fungsi ini menghasilkan tiga orientasi hasil rotasi 90° terhadap sumbu z, x, dan y dengan menukar dua elemen *tuple* orientasi. Hasilnya dipakai oleh operator *rotate* pada kedua mode pembangkitan *neighbor*.

Listing 5: Rotasi 90° terhadap satu sumbu

```
def get_90deg_rotations(orientation: Tuple[str, str, str]) -> List[Tuple[str, str, str
    ]]:

    Generate 3 orientations resulting from a 90-degree rotation around a single axis (x
        , y, or z).

    ox, oy, oz = orientation
    return [
        (oy, ox, oz), # z-axis
        (ox, oz, oy), # x-axis
        (oz, oy, ox), # y-axis
    ]
```

Pembangkitan *neighbor* (`generate_neighbors`). Fungsi ini menerima `max_shift`, `sample_size`, `insert_tries` (jumlah percobaan *insert* per paket di luar truk, bawaan 50), dan `max_sample_attempts` (batas percobaan pada mode *sampling*, bawaan 100). Bagian awal memisahkan paket di dalam dan di luar truk, menyusun himpunan z (lantai dan permukaan atas paket di dalam truk), lalu memilih mode berdasarkan `sample_size`.

Listing 6: `generate_neighbors`: persiapan

```python
    # Neighbor Generation
    def generate_neighbors(
        self,
        truck: Truck,
        package_catalog: Dict[str, Package],
        max_shift: int = 2,
        sample_size: int = None,
        insert_tries: int = 50,
        max_sample_attempts: int = 100
    ) -> List['State']:

        if `sample_size` is given (Stochastic HC), samples actions randomly,
        otherwise (Steepest Ascent / Sideways Move), exhaustively enumerates all swaps,
             moves, single-axis 90° rotations, and up to `insert_tries` insert
            candidates per outside package.

        keys = list(self.placements.keys())
        inside_keys = [k for k in keys if self.placements[k] is not None]
        outside_keys = [k for k in keys if self.placements[k] is None]

        valid_z = {0}
        for pkg_id in inside_keys:
            pos = self.placements[pkg_id]
            pkg = package_catalog[pkg_id]
            _, _, eff_z = pos.get_effective_dimensions(pkg)
            if pos.z + eff_z <= truck.h:
                valid_z.add(pos.z + eff_z)
        z_list = list(valid_z)

        possible_orientations = [
            ('w', 'l', 'h'), ('w', 'h', 'l'),
            ('l', 'w', 'h'), ('l', 'h', 'w'),
            ('h', 'w', 'l'), ('h', 'l', 'w')
        ]
```

Mode *sampling* dipakai bila `sample_size` diberikan. Setiap percobaan memilih satu jenis aksi secara seragam di antara aksi yang tersedia, membangun satu kandidat acak, dan menyimpannya hanya bila valid. Pada *move*, koordinat baru adalah koordinat lama yang digeser acak paling jauh `max_shift` dan dibatasi agar tetap di dalam truk, sedangkan z dipilih acak dari himpunan z. Perulangan berhenti bila telah terkumpul `sample_size` *neighbor* valid atau percobaan mencapai `max_sample_attempts`, sehingga jumlah *neighbor* yang dikembalikan dapat kurang dari `sample_size`. Daftar kandidat tidak dibangun penuh, sehingga seluruh pasangan *swap* tidak perlu dienumerasi.

Listing 7: `generate_neighbors`: mode *sampling*

```python

        # Random sampling (for Stochastic HC)-
        if sample_size is not None:
            available_actions = []
            if len(keys) >= 2 and not (len(inside_keys) == 0 and len(outside_keys) >= 2)
                :
                available_actions.append('swap')
            if inside_keys:
                available_actions.append('move')
                available_actions.append('rotate')
```

```
            if outside_keys:
                available_actions.append('insert')

            if not available_actions:
                return []

            neighbors = []
            attempts = 0
            while len(neighbors) < sample_size and attempts < max_sample_attempts:
                attempts += 1
                action = random.choice(available_actions)
                new_p = self.copy_state().placements

                if action == 'swap':
                    p1, p2 = random.sample(keys, 2)
                    if new_p[p1] is None and new_p[p2] is None:
                        continue
                    new_p[p1], new_p[p2] = new_p[p2], new_p[p1]

                elif action == 'move':
                    pkg_id = random.choice(inside_keys)
                    pos = new_p[pkg_id]
                    pkg = package_catalog[pkg_id]
                    eff_x, eff_y, eff_z = pos.get_effective_dimensions(pkg)
                    max_x = max(0, truck.w - eff_x)
                    max_y = max(0, truck.l - eff_y)
                    nx = max(0, min(max_x, pos.x + random.randint(-max_shift, max_shift)
                        ))
                    ny = max(0, min(max_y, pos.y + random.randint(-max_shift, max_shift)
                        ))
                    nz = random.choice(z_list)
                    if (nx, ny, nz) == (pos.x, pos.y, pos.z):
                        continue
                    new_p[pkg_id] = PackagePlacement(x=nx, y=ny, z=nz, orientation=pos.
                        orientation)

                elif action == 'rotate':
                    pkg_id = random.choice(inside_keys)
                    pos = new_p[pkg_id]
                    ori = random.choice(get_90deg_rotations(pos.orientation))
                    new_p[pkg_id] = PackagePlacement(x=pos.x, y=pos.y, z=pos.z,
                        orientation=ori)

                elif action == 'insert':
                    pkg_id = random.choice(outside_keys)
                    pkg = package_catalog[pkg_id]
                    ori = random.choice(possible_orientations)
                    ex, ey, ez = PackagePlacement(0, 0, 0, ori).get_effective_dimensions
                        (pkg)
                    if ex > truck.w or ey > truck.l or ez > truck.h:
                        continue
                    nx = random.randint(0, truck.w - ex)
                    ny = random.randint(0, truck.l - ey)
                    nz = random.choice(z_list)
                    new_p[pkg_id] = PackagePlacement(x=nx, y=ny, z=nz, orientation=ori)

                candidate = State(placements=new_p)
                if candidate.is_valid(truck, package_catalog):
```

```
                    neighbors.append(candidate)

            return neighbors
```

Mode lengkap dipakai bila `sample_size` tidak diberikan. Seluruh pasangan *swap*, seluruh *move* lokal, tiga rotasi tiap paket, dan kandidat *insert* dikumpulkan lebih dulu. Kandidat *insert* diambil acak sebanyak `insert_tries` percobaan per paket di luar truk dan duplikat dibuang. Setiap kandidat kemudian diterapkan pada salinan *state* dan hanya yang lolos `is_valid` dikembalikan sebagai *neighbor*. Biaya terbesar berasal dari validasi setiap kandidat.

Listing 8: `generate_neighbors`: mode lengkap

```
        # Exhaustive/semi-exhaustive neighborhood (for Steepest & Sideways)
        candidates = []

        # SWAP
        for i in range(len(keys)):
            for j in range(i + 1, len(keys)):
                p1, p2 = keys[i], keys[j]
                if self.placements[p1] is None and self.placements[p2] is None:
                    continue # Both outside, no state change

                candidates.append(('swap', p1, p2))

        for pkg_id in inside_keys:
            pos = self.placements[pkg_id]
            # MOVE
            for nx in range(max(0, pos.x - max_shift), min(truck.w, pos.x + max_shift +
                 1)):
                for ny in range(max(0, pos.y - max_shift), min(truck.l, pos.y +
                    max_shift + 1)):
                    for nz in valid_z:
                        if nx != pos.x or ny != pos.y or nz != pos.z:
                            candidates.append(('move', pkg_id, nx, ny, nz, pos.
                                orientation))

            # ROTATE
            for ori in get_90deg_rotations(pos.orientation):
                candidates.append(('rotate', pkg_id, pos.x, pos.y, pos.z, ori))

        # INSERT
        for pkg_id in outside_keys:
            pkg = package_catalog[pkg_id]
            seen_inserts = set()
            for _ in range(insert_tries):
                ori = random.choice(possible_orientations)
                ex, ey, ez = PackagePlacement(0, 0, 0, ori).get_effective_dimensions(pkg
                    )
                if ex > truck.w or ey > truck.l or ez > truck.h:
                    continue
                nx = random.randint(0, truck.w - ex)
                ny = random.randint(0, truck.l - ey)
                nz = random.choice(z_list)
                key = (pkg_id, nx, ny, nz, ori)
                if key not in seen_inserts:
                    seen_inserts.add(key)
                    candidates.append(('insert', pkg_id, nx, ny, nz, ori))
```

```

        neighbors = []
        for instruction in candidates:
            new_p = self.copy_state().placements

            if instruction[0] == 'swap':
                _, p1, p2 = instruction
                new_p[p1], new_p[p2] = new_p[p2], new_p[p1]
            elif instruction[0] == 'move':
                _, pkg_id, nx, ny, nz, ori = instruction
                new_p[pkg_id] = PackagePlacement(x=nx, y=ny, z=nz, orientation=ori)
            elif instruction[0] == 'rotate':
                _, pkg_id, x, y, z, ori = instruction
                new_p[pkg_id] = PackagePlacement(x=x, y=y, z=z, orientation=ori)
            elif instruction[0] == 'insert':
                _, pkg_id, nx, ny, nz, ori = instruction
                new_p[pkg_id] = PackagePlacement(x=nx, y=ny, z=nz, orientation=ori)

            new_state = State(placements=new_p)
            if new_state.is_valid(truck, package_catalog):
                neighbors.append(new_state)

        return neighbors
```

Pembangkitan *initial state* (`generate_random_state`). Fungsi ini mengimplementasikan rancangan *initial state* acak pada subbagian sebelumnya. Fungsi `parse_input` membaca berkas JSON berisi truk dan daftar paket, lalu mengembalikan objek `Truck` dan katalog paket dengan kunci `id`; atribut `eta` bernilai 0 bila tidak ada pada masukan.

Listing 9: Pembangkitan *initial state* acak

```
def generate_random_state(truck: Truck, package_catalog: Dict[str, Package]) -> State:
    Random initial state (floor)
    # All package is outside of the truck
    placements = {}
    for pkg_id in package_catalog.keys():
        placements[pkg_id] = None

    initial_state = State(placements=placements)

    # Get package ids and shuffle them
    all_pkg_ids = list(package_catalog.keys())
    random.shuffle(all_pkg_ids)

    # 6 possible orientations (w, l, h) mapped to X, Y, Z axes
    possible_orientations = [
        ('w', 'l', 'h'), ('w', 'h', 'l'),
        ('l', 'w', 'h'), ('l', 'h', 'w'),
        ('h', 'w', 'l'), ('h', 'l', 'w')
    ]

    # Try to place packages one by one randomly
    for pkg_id in all_pkg_ids:
        random_ori = random.choice(possible_orientations)

        pkg = package_catalog[pkg_id]
        temp_placement = PackagePlacement(x=0, y=0, z=0, orientation=random_ori)
```

```
        eff_x, eff_y, eff_z = temp_placement.get_effective_dimensions(pkg)

        if eff_z > truck.h:
            continue
        max_x = max(0, truck.w - eff_x)
        max_y = max(0, truck.l - eff_y)

        random_x = random.randint(0, max_x)
        random_y = random.randint(0, max_y)

        # Create placement object
        test_placement = PackagePlacement(
            x=random_x,
            y=random_y,
            z=0,
            orientation=random_ori
        )

        initial_state.placements[pkg_id] = test_placement
        # Check if the placement is valid
        if not initial_state.is_valid(truck, package_catalog):
            initial_state.placements[pkg_id] = None

    return initial_state
```

2.2.3 Kelas `HillClimbing` (`hill_climbing.py`)

Kelas `HillClimbing` menerima objek `Truck` dan katalog paket. Empat metodenya adalah `steepest_ascent`, `sideways_move`, `random_restart`, dan `stochastic`. Pada versi pencatatan yang digunakan untuk eksperimen akhir, `iterations` menghitung setiap percobaan iterasi pencarian, termasuk percobaan terakhir yang membuat algoritma berhenti; `history` menyimpan nilai objektif setelah setiap iterasi, sehingga untuk tiga varian non-restart berlaku `len(history) = iterations + 1`. Pada `random_restart`, riwayat merupakan gabungan beberapa restart dan panjangnya sama dengan total iterasi ditambah jumlah restart. Durasi diukur dengan `time.perf_counter` dan nilai akhir dihitung melalui `get_objective_value`.

Listing 10: Impor dan konstruktor kelas `HillClimbing`

```
import time
import random
from typing import Dict, Tuple, List, Optional

try:
    from src.foundation import State, Truck, Package, generate_random_state
except ModuleNotFoundError:
    from foundation import State, Truck, Package, generate_random_state


class HillClimbing:
    def __init__(self, truck: Truck, catalog: Dict[str, Package]):
        self.truck = truck
        self.catalog = catalog
```

Steepest Ascent Hill-Climbing. Pada setiap iterasi, metode ini membangkitkan kandidat *neighbor* dengan mode lengkap dan memilih nilai tertinggi; jika beberapa kandidat memiliki nilai sama, satu dipilih secara acak. Jika kandidat tidak tersedia atau nilai tertinggi tidak meningkatkan nilai saat ini, algoritma melakukan *resampling* hingga batas `max_resamples` sebelum berhenti. Counter iterasi bertambah pada awal setiap percobaan, bukan hanya ketika perpindahan diterima. Karena penempatan *INSERT* disampling, penghentian setelah beberapa *resampling* tidak membuktikan bahwa state merupakan optimum lokal terhadap semua penempatan yang mungkin.

Listing 11: Pencatatan iterasi Steepest Ascent

```
iterations = 0
history = [current_value]
resample_count = 0

while True:
    iterations += 1
    neighbors = current_state.generate_neighbors(
        self.truck, self.catalog,
        max_shift=2,
        insert_tries=insert_tries,
    )

    if not neighbors:
        history.append(current_value)
        if resample_count < max_resamples:
            resample_count += 1
            continue
        break

    scored = [(n, n.get_objective_value(self.catalog)) for n in neighbors]
    best_value = max(value for _, value in scored)
    best_neighbor = random.choice([
        n for n, value in scored if value == best_value
    ])

    if best_value <= current_value:
        history.append(current_value)
        if resample_count < max_resamples:
            resample_count += 1
            continue
        break

    resample_count = 0
    current_state = best_neighbor
    current_value = best_value
    history.append(current_value)
```

Hill-Climbing with Sideways Move. Varian ini mengizinkan perpindahan ke *neighbor* bernilai sama. Parameter `max_sideways` membatasi jumlah perpindahan mendatar; nilai negatif ditolak dengan `ValueError`. Jika `consecutive=True`, penghitung direset saat ditemukan peningkatan; jika `False`, penghitung dihitung kumulatif. Batas nol hanya melarang perpindahan bernilai sama, tetapi tetap mengizinkan perpindahan yang meningkatkan nilai objektif. Iterasi dihitung di awal loop, termasuk percobaan yang berakhir karena batas sideways tercapai.

Listing 12: Pencatatan iterasi Sideways Move

```
iterations = 0
history = [current_value]
sideways_count = 0
resample_count = 0

while True:
    iterations += 1
    neighbors = current_state.generate_neighbors(
        self.truck, self.catalog,
        max_shift=2,
        insert_tries=insert_tries,
    )

    if not neighbors:
        history.append(current_value)
        if resample_count < max_resamples:
            resample_count += 1
            continue
        break

    scored = [(n, n.get_objective_value(self.catalog)) for n in neighbors]
    best_value = max(value for _, value in scored)
    best_neighbor = random.choice([
        n for n, value in scored if value == best_value
    ])

    if best_value < current_value:
        history.append(current_value)
        if resample_count < max_resamples:
            resample_count += 1
            continue
        break
    elif best_value == current_value:
        if sideways_count >= max_sideways:
            history.append(current_value)
            break
        sideways_count += 1
        is_sideways = True
    else:
        if consecutive:
            sideways_count = 0
        is_sideways = False

    resample_count = 0
    current_state = best_neighbor
    current_value = best_value
    history.append(current_value)

    if is_sideways and sideways_count >= max_sideways:
        break
```

Random Restart Hill-Climbing. Fungsi ini memanggil `steepest_ascent` sebanyak `max_restart` kali, masing-masing dari *initial state* acak yang baru, dan mengembalikan hasil terbaik dari seluruh *restart*. Nilai `max_restart` harus positif, jika tidak akan memunculkan `ValueError`. Selain keluaran umum, fungsi mengembalikan `restart_iterations`

(banyak iterasi tiap *restart*) dan `restart_indices` (posisi awal tiap *restart* pada `history` gabungan), sehingga banyak *restart* sama dengan panjang `restart_iterations` dan plot dapat menandai batas antar-*restart*. Iterasi yang dilaporkan adalah total seluruh *restart*, dan durasi mencakup seluruh *restart*.

Listing 13: *Random Restart Hill-Climbing*

```
    def random_restart(
        self,
        max_restart: int,
        insert_tries: int = 50,
        max_resamples: int = 3
    ) -> Tuple[State, State, int, float, List[int], List[int], List[int]]:

        Random Restart Hill-Climbing
        Executes Steepest Ascent multiple times from entirely randomized starting
            positions.

        if max_restart <= 0:
            raise ValueError(fmax_restart must be a positive integer (> 0), got: {
                max_restart})

        start_time = time.perf_counter()

        global_best_state = None
        global_best_start_state = None
        global_best_value = -1
        global_history = []
        restart_iterations = []
        restart_indices = []
        total_iterations = 0

        for i in range(max_restart):
            restart_indices.append(len(global_history))

            state, start_state, iters, _, history = self.steepest_ascent(
                initial_state=None,
                insert_tries=insert_tries,
                max_resamples=max_resamples
            )
            val = state.get_objective_value(self.catalog)

            restart_iterations.append(iters)
            total_iterations += iters
            global_history.extend(history)

            if val > global_best_value:
                global_best_value = val
                global_best_state = state
                global_best_start_state = start_state

        duration = time.perf_counter() - start_time
        return global_best_state, global_best_start_state, total_iterations, duration,
            global_history, restart_iterations, restart_indices
```

Stochastic Hill-Climbing. Varian ini tidak mengevaluasi seluruh *neighbor*. Pada setiap percobaan, `generate_neighbors` dipanggil dengan `sample_size` (bawaan 15) se-

hingga *neighbor* dibangkitkan secara langsung dengan mode *sampling*, tanpa membangun daftar kandidat penuh. *Neighbor* dalam sampel yang bernilai lebih tinggi dari nilai saat ini dikumpulkan, lalu satu di antaranya dipilih acak. Bila tidak ada yang lebih baik, `no_improve` bertambah, dan pencarian berhenti setelah `max_no_improve` percobaan gagal berturut-turut (bawaan 50). Parameter `max_sample_attempts` (bawaan 100) membatasi percobaan pembangkitan sampel. Banyak iterasi dan `history` hanya menghitung perpindahan yang diterima, bukan jumlah percobaan.

Listing 14: Pencatatan iterasi Stochastic Hill-Climbing

```
history = [current_value]
iterations = 0
no_improve = 0

while no_improve < max_no_improve:
    iterations += 1
    neighbors = current_state.generate_neighbors(
        self.truck, self.catalog,
        max_shift=2,
        sample_size=sample_size,
        max_sample_attempts=max_sample_attempts,
    )
    better_neighbors = [
        n for n in neighbors
        if n.get_objective_value(self.catalog) > current_value
    ]

    if not better_neighbors:
        no_improve += 1
        history.append(current_value)
        continue

    current_state = random.choice(better_neighbors)
    current_value = current_state.get_objective_value(self.catalog)
    history.append(current_value)
    no_improve = 0
```

2.3 Implementasi Simulated Annealing

2.3.1 Kelas dan pembangkitan tetangga

Kelas `SimulatedAnnealing` menerima objek truk, katalog paket, dan fungsi objektif. Jika tidak diberikan fungsi objektif alternatif, algoritma memakai jumlah `value` paket yang berada di dalam truk. Fungsi `random_neighbor` mengambil satu jenis aksi secara acak dari `swap`, `move`, `rotate`, dan `insert`; kandidat dicoba hingga batas percobaan tercapai dan hanya dikembalikan apabila `State.is_valid` menyatakan state valid. Jika tidak ditemukan kandidat valid, iterasi tetap dicatat dan temperatur tetap diturunkan.

Operator `rotate` harus memakai `get_90deg_rotations` agar satu aksi hanya memutar paket 90° terhadap satu sumbu. Pemilihan sembarang dari lima orientasi lain tidak setara dengan satu rotasi tersebut, sebab dapat mengubah orientasi melalui lebih dari satu rotasi dasar.

Listing 15: Operator rotasi pada SA

```
elif kind == rotate:
    inside = [pid for pid in ids if new_p[pid] is not None]
    if not inside:
        continue
    pid = random.choice(inside)
    pos = new_p[pid]
    ori = random.choice(get_90deg_rotations(pos.orientation))
    new_p[pid] = PackagePlacement(pos.x, pos.y, pos.z, ori)
```

2.3.2 Aturan penerimaan dan penyimpanan hasil

Karena persoalan merupakan maksimisasi, perubahan nilai dihitung sebagai

$$\Delta E = f(s_{\text{neighbor}}) - f(s_{\text{current}}).$$

Jika $\Delta E \geq 0$, tetangga selalu diterima. Jika $\Delta E < 0$, tetangga diterima dengan probabilitas $\exp(\Delta E/T)$. Temperatur awal adalah `t0`, kemudian dikalikan faktor pendinginan `alpha` pada setiap iterasi. Algoritma berhenti ketika mencapai `max_iter` atau temperatur tidak lagi lebih besar daripada `t_min`. Implementasi menyimpan `final_state` dan `best_state` secara terpisah karena SA dapat menerima state yang lebih buruk setelah sebelumnya menemukan state dengan nilai lebih tinggi.

Listing 16: Inti aturan penerimaan Simulated Annealing

```
nb_val = objective_fn(neighbor)
delta = nb_val - current_value
accept = delta >= 0

if delta < 0:
    probability = math.exp(delta / temperature)
    if random.random() < probability:
        accept = True
        accepted_worse += 1
    else:
        rejected_worse += 1

if accept:
    current = neighbor
    current_value = nb_val
    if current_value > best_value:
        best_state = current.copy_state()
        best_value = current_value
```

Definisi metrik stagnasi. Penolakan tetangga yang lebih buruk adalah perilaku normal SA dan bukan bukti bahwa algoritma berada di optimum lokal. Oleh karena itu, laporan membedakan (1) `rejected_worse_count`, jumlah tetangga bernilai lebih rendah yang ditolak; (2) `worse_neighbor_rejection_rate`, yaitu jumlah penolakan dibagi total kandidat lebih buruk yang diterima atau ditolak; dan (3) `non_improving_iteration_frequency`, yaitu proporsi iterasi ketika kandidat tidak meningkatkan objective terhadap state saat itu. Metrik ketiga merupakan indikator stagnasi berdasarkan satu tetangga acak, bukan bukti local optimum eksak. Jika tidak ada kandidat yang lebih buruk, rejection rate tidak terdefinisi dan ditulis N/A, bukan 0%.

2.4 Implementasi Genetic Algorithm

2.4.1 Representasi individu dan fitness

Kelas `GeneticAlgorithm` merepresentasikan setiap individu sebagai pasangan `(state, fitness)`. Populasi awal dibentuk dengan memanggil `generate_random_state`. Pada eksperimen utama, fungsi objektif yang digunakan adalah `base`, yaitu penjumlahan nilai (`value`) seluruh paket yang berada di dalam truk. Sebelum fitness dihitung, setiap individu diperiksa untuk memastikan bahwa pemetaan paket lengkap dan *state* memenuhi validasi `State.is_valid`.

Setelah perbaikan validitas GA, penghitungan fitness dipisahkan melalui satu titik pemeriksaan. Hal ini penting terutama untuk objective `base`, yang hanya menjumlahkan value dan tidak dengan sendirinya memberikan penalti terhadap placement yang tidak valid.

Listing 17: Gerbang validasi sebelum fitness dihitung

```
def _evaluate_fitness(self, state):
    if set(state.placements) != set(self.catalog):
        raise ValueError(Mapping paket tidak lengkap)
    if not state.is_valid(self.truck, self.catalog):
        raise ValueError(State GA tidak valid)
    return float(self.objective_fn(state))
```

2.4.2 Seleksi, crossover, mutasi, dan replacement

Seleksi parent menggunakan *roulette-wheel selection*: probabilitas pemilihan ditentukan oleh fitness yang telah digeser agar bobot tidak negatif. Crossover satu titik mengambil gen dari dua parent dan membentuk kandidat child. Hasil gabungan tidak otomatis valid, sebab dua paket yang berasal dari parent berbeda dapat bertumpang-tindih atau kehilangan dukungan. Oleh karena itu, child divalidasi dan diperbaiki sebelum fitness dihitung; kandidat yang tidak dapat ditempatkan secara valid tetap berada di luar truk. Perbaikan ini memprioritaskan validitas, tetapi sifatnya greedy dan tidak menjamin child bernilai maksimum.

Mutasi mengambil salah satu neighbor valid dari `State.generate_neighbors` ketika peluang mutasi terpenuhi. Pada setiap generasi, parent dan child digabungkan, diurutkan menurut fitness, dan sejumlah `population_size` individu terbaik dipertahankan (strategi $\mu + \lambda$). Program menyimpan riwayat fitness maksimum dan rata-rata populasi pada setiap generasi untuk visualisasi.

Listing 18: Inti replacement Genetic Algorithm

```
selected_parents = self.selected_parents()
children = [
    self.mutate(child) if random.random() < self.mutation_rate else child
    for parent1, parent2 in selected_parents
    for child in [self.crossover(parent1, parent2)]
]

new_population = self.population + children
new_population.sort(key=lambda individual: individual.fitness,
                    reverse=True)
```

```
self.population = new_population[:self.population_size]
_record_population(max_history, avg_history)
```

2.5 Visualisasi State dan Hasil Pencarian

Fungsi `draw_state` menggambarkan ruang peti kemas dalam plot 3D dan memberi label ID pada paket yang terangkut. Paket *fragile* dibedakan dengan garis tepi merah yang lebih tebal. Fungsi `plot_initial_final` memperlihatkan state awal dan akhir secara berdampingan, sedangkan panel informasi menampilkan objective total, berat terangkut, dimensi truk, jumlah paket di dalam dan di luar truk, serta rincian setiap paket di luar truk: ID, dimensi asli, value, berat, ETA, dan status fragile. Tinggi panel informasi perlu menyesuaikan jumlah paket luar agar seluruh rincian dapat dibaca, bukan berhenti pada sejumlah paket pertama.

Grafik HC dan SA menampilkan objective terhadap iterasi; untuk Random Restart, kurva setiap restart harus dipisahkan atau diberi penanda batas agar reset state awal tidak disalahartikan sebagai penurunan objective di dalam satu pencarian. Grafik SA probabilitas hanya mempunyai titik ketika neighbor dengan objective lebih rendah diajukan. Grafik GA menampilkan objective maksimum dan rata-rata populasi pada setiap generasi, sebagaimana diperlukan untuk menganalisis pengaruh ukuran populasi dan banyak generasi.

2.6 Hasil dan Analisis Eksperimen

2.6.1 Metodologi

Eksperimen berikut menggunakan masukan `input/sample_input.json`. Runner dijalankan sebanyak tiga kali untuk setiap algoritma atau varian, dengan seed dasar 42. Seed per-run diubah oleh runner menggunakan `seed + nomor_run`. Objective dasar memaksimalkan jumlah `value` paket yang terangkut; solusi yang dibandingkan harus memenuhi batas dimensi, kapasitas berat, tidak tumpang-tindih, memiliki support, dan tidak menggunakan paket fragile sebagai penopang.

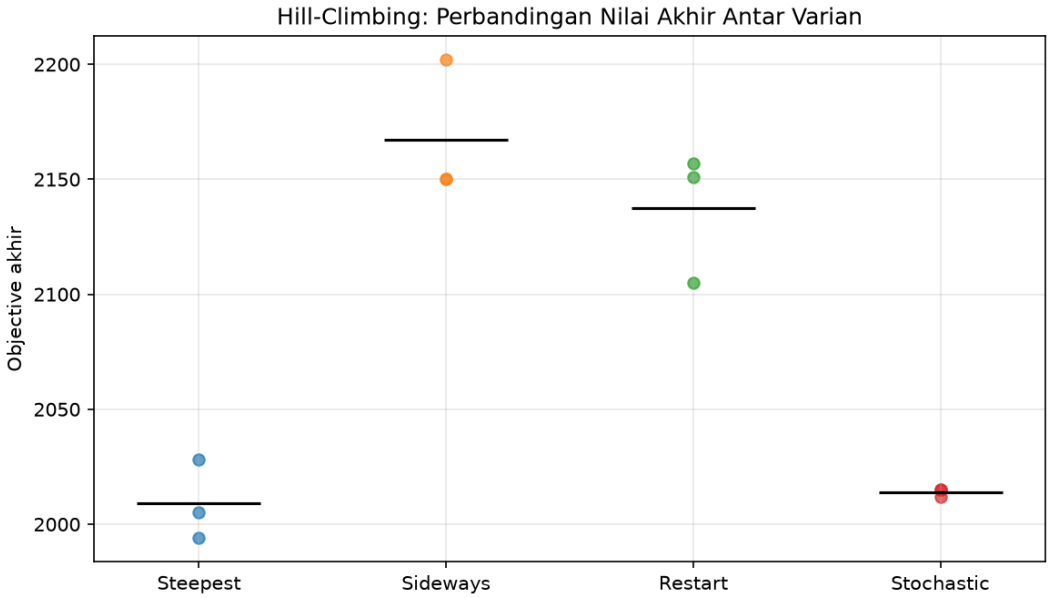
2.6.2 Hill-Climbing

Tabel 1 merangkum tiga run pada setiap varian HC dari artefak yang tersedia. Nilai yang ditampilkan adalah total value paket yang terangkut.

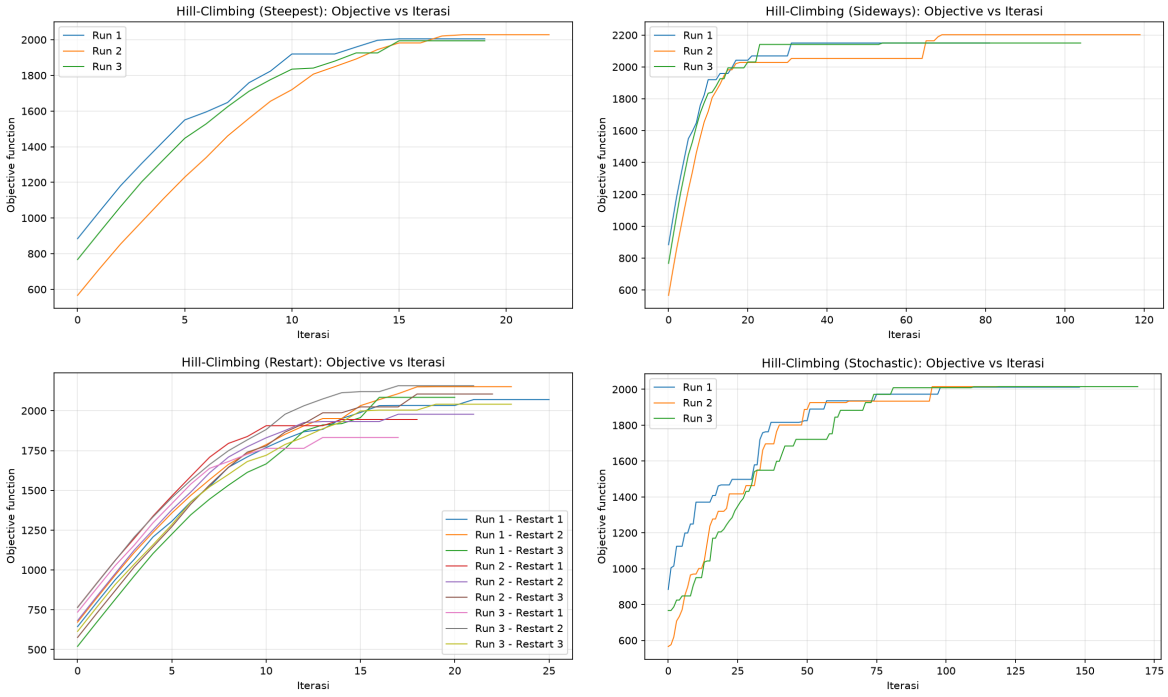
Tabel 1: Hasil baseline Hill-Climbing (tiga run per varian)

Varian	Nilai awal (R1/R2/R3)	Nilai akhir (R1/R2/R3)	Rata-rata akhir	Iterasi rata-rata*	Durasi rata-rata (s)
Steepest Ascent	884/566/767	2005/2028/1994	2009,0	14,7	4,822
Sideways Move	884/566/767	2150/2202/2150	2167,3	101,3	39,262
Random Restart	672/576/764	2151/2105/2157	2137,7	45,3	12,454
Stochastic	884/566/767	2012/2015/2015	2014,0	27,0	1,733

Rata-rata nilai akhir tertinggi di antara varian HC yang tersedia diperoleh Sideways Move (2167,3), diikuti Random Restart (2137,7), Stochastic (2014,0), dan Steepest Ascent



Gambar 1: Perbandingan nilai akhir antarvarian Hill-Climbing pada artefak baseline.



Gambar 2: Riwayat objective empat varian Hill-Climbing.

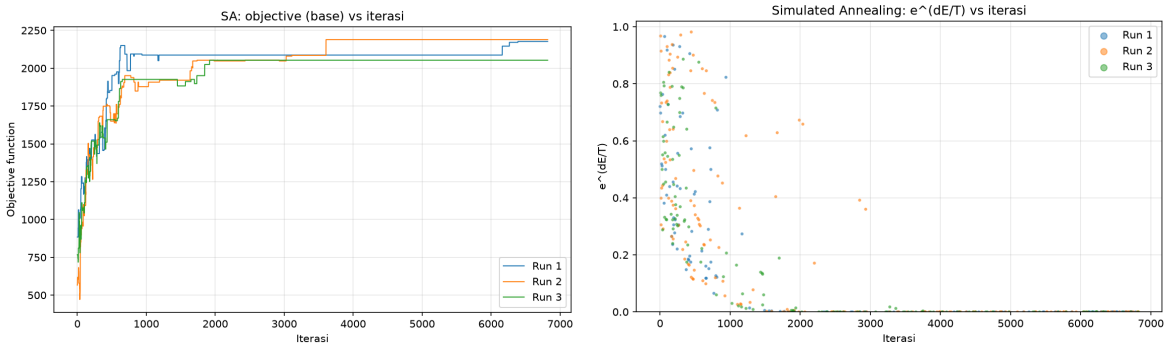
(2009,0). Sideways Move membutuhkan waktu paling lama, sekitar 39,262 detik, karena kandidat bernilai sama dapat diterima dan pencarian berlangsung lebih panjang. Stochastic paling cepat pada baseline, sekitar 1,733 detik, tetapi hasil akhirnya lebih rendah daripada Sideways Move dan Random Restart. Random Restart meningkatkan peluang menemukan solusi awal yang lebih baik dengan menjalankan Steepest Ascent berulang kali; biaya waktunya lebih besar daripada satu run Steepest Ascent. Hasil ini hanya membandingkan tiga run dan tidak membuktikan nilai optimum global.

2.6.3 Simulated Annealing

Eksperimen SA baseline menggunakan $T_0 = 91{,}8667$, $\alpha = 0{,}999$, $T_{\min} = 0{,}1$, dan batas maksimum 20000 iterasi. Implementasi berhenti setelah 6820 iterasi karena temperatur turun mencapai kondisi berhenti, sehingga jumlah iterasi aktual lebih kecil daripada batas maksimumnya.

Tabel 2: Hasil baseline Simulated Annealing

Objective	Nilai awal (R1/R2/R3)	Nilai akhir objective (R1/R2/R3)	Rata-rata durasi (s)	Iterasi aktual
Base	884/566/767	2189/2189/2189	13,241	6820
ETA (bonus)	884/566/767	2775,524/2775,524/2775,524	13,039	6820



Gambar 3: Riwayat objective dan probabilitas penerimaan kandidat lebih buruk pada SA dengan objective dasar

Ketiga run dengan objective dasar mencapai nilai akhir 2189, dengan durasi rata-rata 13,241 detik. Kesamaan nilai akhir pada ketiga run menunjukkan konsistensi hasil pada data uji ini, tetapi tidak membuktikan tercapainya optimum global. Pada objective ETA, skor objektif mencapai 2775,524, sedangkan nilai dasar paket yang terangkut tetap 2189 pada ketiga run. Skor objective ETA tidak dapat dibandingkan langsung dengan skor objective dasar karena keduanya menggunakan formula penilaian yang berbeda. Ringkasan lama masih memuat metrik `stuck_count`, yang menghitung penolakan terhadap kandidat bernilai lebih buruk. Metrik tersebut tidak digunakan untuk menyimpulkan bahwa algoritma berada di *local optimum*. Evaluasi perilaku Simulated Annealing menggunakan tiga metrik yang telah dihitung, yaitu `rejected_worse_count`, `worse_neighbor_rejection_rate`, dan `non_improving_iteration_frequency`, untuk membedakan frekuensi penolakan kandidat yang lebih buruk dari indikasi stagnasi pencarian.

2.6.4 Genetic Algorithm

Eksperimen Genetic Algorithm (GA) menggunakan fungsi objektif dasar `base`, yaitu memaksimalkan jumlah `value` paket yang berhasil dimuat. Laju mutasi ditetapkan sebesar 0,01 dan setiap konfigurasi dijalankan tiga kali. Dua rangkaian eksperimen dilakukan agar pengaruh jumlah generasi dan ukuran populasi dapat dibandingkan secara terpisah. Pada rangkaian A, ukuran populasi dijaga tetap 100, sedangkan jumlah generasi divariasikan menjadi 50, 100, dan 150. Pada rangkaian B, jumlah generasi dijaga tetap 100, sedangkan ukuran populasi divariasikan menjadi 25, 50, dan 75.

Nilai awal pada tabel merupakan nilai individu terbaik dalam populasi awal masing-masing run, sedangkan nilai akhir merupakan nilai objektif solusi terbaik pada akhir generasi. Seluruh 18 *final state* pada keenam konfigurasi diperiksa kembali menggunakan `State.is_valid`; semuanya valid. Nilai `final_base_value` pada ringkasan juga cocok dengan perhitungan ulang total value paket dalam state akhir.

Tabel 3: Hasil GA dengan ukuran populasi tetap 100 dan variasi jumlah generasi

Generasi	Populasi	Nilai awal (R1/R2/R3)	Nilai akhir (R1/R2/R3)	Rata-rata nilai akhir	Rata-rata peningkatan	Durasi rata-rata (s)
50	100	1175/996/1019	1417/1619/1467	1501,0	437,7	1,530
100	100	1175/996/1019	1537/1747/1536	1606,7	543,3	3,530
150	100	1175/996/1019	1625/1861/1645	1710,3	647,0	6,121

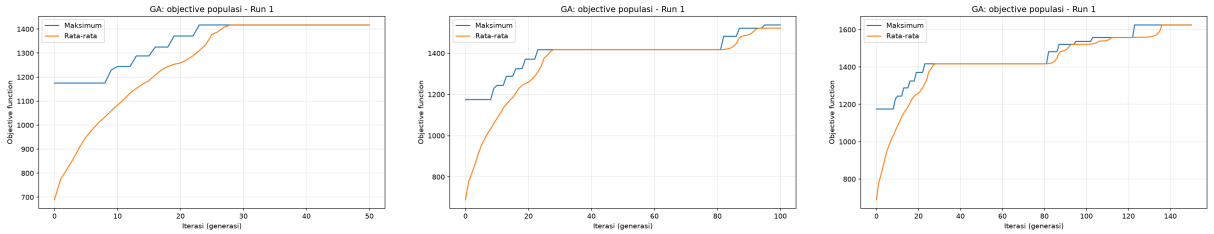
Tabel 4: Hasil GA dengan jumlah generasi tetap 100 dan variasi ukuran populasi

Generasi	Populasi	Nilai awal (R1/R2/R3)	Nilai akhir (R1/R2/R3)	Rata-rata nilai akhir	Rata-rata peningkatan	Durasi rata-rata (s)
100	25	906/993/846	1128/1102/1313	1181,0	266,0	0,564
100	50	1147/993/980	1455/1290/1416	1387,0	347,0	1,226
100	75	1175/993/1019	1410/1410/1329	1383,0	320,7	2,153

Analisis pengaruh jumlah generasi. Pada ukuran populasi tetap 100, rata-rata nilai akhir meningkat seiring bertambahnya generasi: dari 1501,0 pada 50 generasi menjadi 1606,7 pada 100 generasi dan 1710,3 pada 150 generasi. Dengan demikian, konfigurasi 150 generasi menghasilkan rata-rata nilai akhir tertinggi pada rangkaian ini, dengan peningkatan rata-rata 209,3 dibandingkan konfigurasi 50 generasi. Peningkatan tersebut disertai kenaikan durasi rata-rata dari 1,530 detik menjadi 6,121 detik. Hasil ini menunjukkan bahwa tambahan generasi memberi kesempatan bagi populasi untuk menghasilkan solusi dengan value lebih tinggi, tetapi membutuhkan waktu komputasi lebih lama. Kesimpulan ini terbatas pada tiga run untuk setiap konfigurasi dan tidak membuktikan tercapainya optimum global.

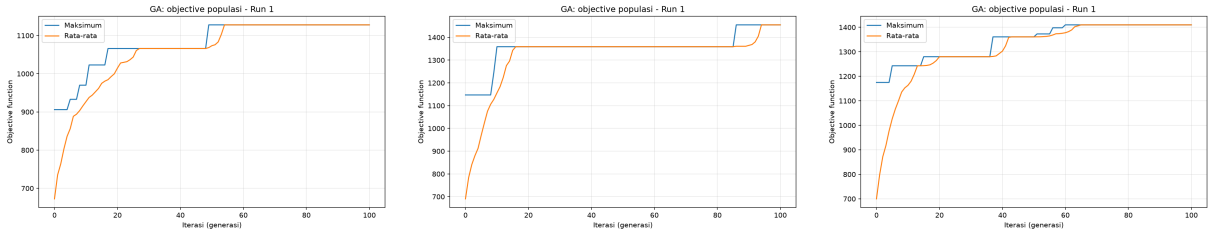
Analisis pengaruh ukuran populasi. Pada 100 generasi, rata-rata nilai akhir meningkat dari 1181,0 pada populasi 25 menjadi 1387,0 pada populasi 50. Ketika populasi dinaikkan menjadi 75, rata-rata nilai akhir menjadi 1383,0, sedikit lebih rendah daripada populasi 50. Jadi, pada eksperimen ini, memperbesar populasi dari 25 ke 50 memberikan peningkatan rata-rata yang jelas, sedangkan kenaikan dari 50 ke 75 tidak meningkatkan rata-rata solusi. Durasi rata-rata juga meningkat seiring ukuran populasi, dari 0,564 detik menjadi 1,226 detik dan 2,153 detik. Hasil tiga run ini belum cukup untuk menyimpulkan bahwa populasi 50 selalu lebih baik; hasil GA bersifat stokastik dan dapat berubah dengan seed maupun parameter lainnya.

Perbandingan nilai awal antarukuran populasi perlu dibaca dengan hati-hati. Nilai awal yang dicatat merupakan individu terbaik dari populasi awal, bukan satu state awal identik yang digunakan pada semua konfigurasi. Karena itu, perubahan nilai akhir mencerminkan gabungan kualitas populasi awal dan perkembangan selama evolusi. Meski demikian, seluruh run pada tabel menghasilkan nilai akhir lebih tinggi daripada nilai awal masing-masing.



(a) Populasi 100, 50 generasi (b) Populasi 100, 100 generasi (c) Populasi 100, 150 generasi

Gambar 4: Kurva objective maksimum dan rata-rata populasi pada run pertama untuk rangkaian variasi jumlah generasi.



(a) Populasi 25, 100 generasi (b) Populasi 50, 100 generasi (c) Populasi 75, 100 generasi

Gambar 5: Kurva objective maksimum dan rata-rata populasi pada run pertama untuk rangkaian variasi ukuran populasi.

Kedua grafik memperlihatkan perkembangan objective maksimum dan rata-rata populasi sepanjang generasi untuk satu run representatif pada tiap konfigurasi. Interpretasi kuantitatif tetap didasarkan pada ketiga run di setiap konfigurasi seperti dirangkum dalam Tabel 3 dan Tabel 4; kurva satu run tidak digunakan sebagai pengganti ringkasan tiga run.

Analisis lintas algoritma. Pada hasil baseline yang tercatat sebelumnya, Simulated Annealing dengan objective dasar menghasilkan rata-rata nilai akhir 2189, sedangkan rata-rata varian Hill-Climbing berada pada rentang 2009,0–2167,3. Pada enam konfigurasi GA yang diuji, rata-rata nilai akhir tertinggi adalah 1710,3 pada populasi 100 dan 150 generasi. Angka tersebut merupakan hasil konfigurasi GA yang diuji, bukan bukti bahwa GA secara umum lebih buruk daripada HC atau SA. Batas iterasi dan biaya pencarian masing-masing algoritma berbeda, sehingga perbandingan lintas algoritma ini bersifat deskriptif, bukan perbandingan dengan anggaran komputasi yang setara. Hasil juga terbatas pada tiga run per konfigurasi.

Validasi dan batasan analisis. Seluruh 18 state akhir GA pada hasil eksperimen yang dianalisis lolos validasi constraint: batas dimensi truk, kapasitas berat, tidak ada tumpang tindih, setiap paket bertumpu secara valid, dan paket *fragile* tidak menjadi penopang. Nilai akhir juga sesuai dengan perhitungan ulang objective dasar dari state.

Eksperimen belum membuktikan optimum global; eksperimen hanya menunjukkan bahwa pada konfigurasi ini, penambahan generasi meningkatkan rata-rata nilai akhir, sedangkan peningkatan populasi di atas 50 tidak memberi kenaikan rata-rata tambahan pada 100 generasi.

3 Kesimpulan dan Saran

3.1 Kesimpulan

Tujuan tugas besar ini telah tercapai. Program mampu merepresentasikan persoalan pemuatan paket ke dalam peti kemas truk, memeriksa seluruh *hard constraint* (batas dinding, tumpang tindih, *support*, barang *fragile*, dan kapasitas berat), serta membangkitkan *neighbor* melalui operator *swap*, *move*, *rotate*, dan *insert*. Tiga kelompok algoritma *local search* berhasil diimplementasikan, yaitu *Hill-Climbing* (keempat varian: *Steepest Ascent*, *Sideways Move*, *Random Restart*, dan *Stochastic*), *Simulated Annealing*, serta *Genetic Algorithm*. Seluruh *final state* yang dilaporkan pada eksperimen GA lolos pemeriksaan `State.is_valid`. Selain itu, program menyediakan visualisasi 3D untuk *initial state* dan *final state*, serta grafik objektif terhadap iterasi. Implementasi keempat varian *Hill-Climbing* dan fungsi objektif alternatif berbasis ETA juga telah dikerjakan sebagai bagian bonus.

Berdasarkan eksperimen dengan tiga kali percobaan pada setiap algoritma atau konfigurasi, diperoleh kesimpulan berikut.

1. **Kualitas solusi.** *Simulated Annealing* memiliki rata-rata nilai akhir tertinggi (2189), diikuti *Sideways Move* (2167,3), *Random Restart* (2137,7), *Stochastic* (2014,0), dan *Steepest Ascent* (2009,0). Konfigurasi GA terbaik (populasi 100, 150 generasi) mencapai 1710,3. Operator *move*, *rotate*, dan *swap* di dalam truk tidak mengubah nilai objektif, sehingga *Steepest Ascent* dan *Stochastic* mudah berhenti di area datar. Varian lain dapat melewatinya melalui langkah mendatar, *restart*, atau penerimaan langkah yang lebih buruk. Penjelasan ini masih dugaan dan belum diuji terpisah.

2. **Durasi pencarian.** *Stochastic* paling cepat (1,733 detik) dan *Sideways Move* paling lama (39,262 detik), sedangkan SA membutuhkan 13,241 detik. GA berjalan 0,564–6,121 detik dengan hasil lebih rendah. Dengan demikian terdapat *trade-off* antara waktu komputasi dan kualitas solusi.

3. **Konsistensi hasil.** SA menghasilkan nilai yang sama (2189) pada ketiga *run*, varian HC bervariasi kecil, dan GA paling bervariasi (misalnya 1537–1747 pada populasi 100 dengan 100 generasi) karena proses seleksi, *crossover*, dan mutasi bersifat acak.

4. **Pengaruh parameter GA.** Pada populasi 100, rata-rata nilai akhir naik dari 1501,0 (50 generasi) ke 1606,7 (100 generasi) dan 1710,3 (150 generasi). Pada 100 generasi, populasi 25 menghasilkan 1181,0, populasi 50 menghasilkan 1387,0, dan populasi 75 menghasilkan 1383,0 sehingga tidak ada peningkatan lanjutan. Durasi bertambah seiring generasi dan ukuran populasi.

Perbandingan antaralgoritma bersifat deskriptif karena kriteria berhenti dan anggaran komputasi tiap algoritma berbeda, hasil hanya berasal dari tiga *run* pada satu berkas

masukan, dan hasil GA hanya berlaku untuk konfigurasi yang diuji. Hasil ini tidak menunjukkan keunggulan mutlak suatu algoritma.

3.2 Saran

1. **Pembanding optimum global.** Hitung batas atas atau solusi eksak untuk instans kecil (misalnya dengan *integer linear programming*) agar kedekatan tiap algoritma dapat diukur.

2. **Perbaikan *neighbor*.** Pandu operator *insert* dengan titik sudut paket yang sudah ada (*extreme points*) alih-alih posisi acak, dan tambahkan pembeda sekunder seperti kekompakan penempatan untuk membedakan *neighbor* bernilai sama.

3. **Penyetelan parameter.** Teliti pengaruh T_0, α, dan jadwal pendinginan pada SA, serta laju mutasi, *elitism*, dan jumlah generasi di atas 150 pada GA.

4. **Efisiensi validasi.** `State.is_valid` berbiaya kuadratik dan dipanggil untuk setiap kandidat, sehingga validasi inkremental yang hanya memeriksa paket yang berubah dapat mempercepat seluruh algoritma.

4 Lampiran

4.1 Lampiran Form Penggunaan Generative AI

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, DeepL, Quillbot, Grammarlybot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Ya	2
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Tidak	0
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	1
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	0
Lainnya, Jelaskan:		

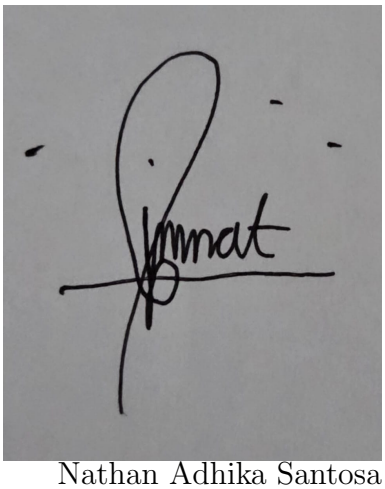
Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 20 Oktober 2026



Muhammad Iqbal Raihan

13524011



Nathan Adhika Santosa

13524041



Muhammad Haris Putra Sulastianto

13524053

4.2 Pranala Ke Repositori

https://github.com/mhps-null/IF3170_G26_150cc

4.3 Pembagian Tugas

NIM	Nama	Tugas
13524011	Muhammad Iqbal Raihan	Genetic Algorithm dan Bonus 3
13524041	Nathan Adhika Santosa	Laporan, Fondasi (objective function, validasi, neighbor, input parser), dan Hill-Climbing (semua varian)
13524053	Muhammad Haris Putra Sulastianto	Laporan, Simulated Annealing, visualisasi, dan Bonus 1 (objective function alternatif)

Tabel 5: Pembagian Tugas Kelompok

Pustaka

[1] Asisten Laboratorium AI IF3170. Spesifikasi tugas besar 1 inteligensi artifisial 2026/2027. Dokumen spesifikasi tugas, Institut Teknologi Bandung, 2026.

[2] M. L. Khodra. Beyond classical search: Classical vs local search. YouTube, 2021. Accessed: September 16, 2026. URL: https://www.youtube.com/watch?v=oGTa9kGE3P8.

[3] M. L. Khodra. Beyond classical search: Genetic algorithm. YouTube, 2021. Accessed: September 16, 2026. URL: https://www.youtube.com/watch?v=aI41TGo9J9c.

[4] M. L. Khodra. Beyond classical search: Hill-climbing search. YouTube, 2021. Accessed: September 16, 2026. URL: https://www.youtube.com/watch?v=L0b0CN3Fds8.

[5] M. L. Khodra. Beyond classical search: Simulated annealing. YouTube, 2021. Accessed: September 16, 2026. URL: https://www.youtube.com/watch?v=AdhG7HnhYrk.

[6] M. L. Khodra. Beyond classical search: State (value, neighbor). YouTube, 2021. Accessed: September 16, 2026. URL: https://www.youtube.com/watch?v=_6rla37QorM.

[7] Tim Pengajar IF3170. Salindia perkuliahan inteligensi artifisial. Materi perkuliahan IF3170 Inteligensi Artifisial, Institut Teknologi Bandung, 2026.